

10 — Recursion & Backtracking

DSA in Python — Interview Prep

Recursion & Backtracking — Tutorial

Covers the 10 problems solved in this folder, opening Phase 2 of the syllabus (Algorithmic Patterns) after Phase 1's 9 core data structure topics. Every single problem here uses the exact same three-step shape:

```
def backtrack(state):
    if base_case:
        record_or_return
    for choice in options(state):
        make_choice(choice)      # choose
        backtrack(next_state)    # explore
        undo_choice(choice)      # un-choose
```

What changes between problems is **what counts as a choice**, **what the base case is**, and **what pruning rule skips bad choices early**. Once that shape is automatic, "backtracking problem" stops being intimidating and becomes a fill-in-the-blanks exercise.

The patterns

1. Record at every node vs. only at the leaves (01/02 vs 03)

Subsets are valid at **every** partial length — `[1]` and `[1, 2]` are both legitimate subsets of `[1, 2, 3]` — so `subsets` records `path` into the result at the top of every call, before the loop even runs. Permutations are only valid once **complete** — a partial arrangement isn't a permutation of anything — so `permutations` only records at the base case where `len(path) == len(nums)`. Knowing which of these two a problem needs is often the first fork in the road.

2. start index vs. used array (01/02 vs 03)

Subsets and combinations care about **combinations**, not order, so a `start` index ensures each element is only chosen after ones already in the path — this alone prevents `[1, 2]` and `[2, 1]` from both appearing. Permutations care about order, so instead of a `start` index there's a `used` boolean array — every not-yet-used element is a valid next choice regardless of position.

3. Skipping duplicates at the same tree level (02, 05)

When the input has duplicate values but the output must not have duplicate results, sort first (so equal values become adjacent), then skip a candidate if it equals the **previous** candidate at the same level of the loop (`i > start` and `nums[i] == nums[i-1]`). The `i > start` guard is the whole trick: it allows two equal values to be used **together** within one path (that's fine, produces a valid combination), while blocking two equal values from each **starting** a sibling branch at the same level (that would just regenerate the same combination twice).

4. Reuse allowed vs. not allowed (04 vs 05)

Both are "find combinations summing to target," differing in one line: `combination_sum` recurses with `backtrack(i, ...)`, allowing the same index to be chosen again immediately (reuse); `combination_sum_ii` recurses with `backtrack(i + 1, ...)`, moving strictly forward (no reuse). Small change, and worth stating out

loud in an interview exactly *why* that one index matters.

5. Pruning as soon as a choice is provably bad (04, 05)

`combination_sum` prunes the instant `remaining < 0` — no point exploring further down a branch that already overshot the target. `combination_sum_ii` goes further: because candidates are sorted, the moment `candidates[i] > remaining`, *every later* candidate is also too big, so the loop can `break` entirely rather than just skip one option — pruning a whole remaining sub-branch at once.

6. Counters instead of an index for the base case (06)

`generate_parentheses` doesn't backtrack over a fixed input array at all — the "choices" are just "add an open paren" or "add a close paren," gated by two counters (`open_count < n`, `close_count < open_count`) instead of a `start/used` structure. The second gate is what keeps every *prefix* valid, not just the finished string — worth noticing this is a stronger guarantee than checking validity only at the end.

7. Pure Cartesian product, no pruning (07)

`letter_combinations` has no constraint to check at all — every letter for the current digit is always a valid next choice. It's included specifically to show that backtracking's value isn't always about pruning; sometimes it's just a clean way to generate a nested Cartesian product without knowing the nesting depth ahead of time.

8. Backtracking over where to cut, not what to pick (08)

`palindrome_partitioning` chooses *substring boundaries* rather than elements from a fixed array — at each position, every possible end point for the next piece is a candidate choice, filtered by whether that specific substring is a palindrome. The base case is reaching the end of the string with every character assigned to some valid piece.

9. O(1) constraint checks via auxiliary sets (09)

N-Queens is the first problem here needing to check a *derived* property (does this placement conflict with any earlier one) efficiently. Placing one queen per row makes row-conflicts structurally impossible by construction; column and diagonal conflicts are checked in O(1) via sets keyed by the invariant that's constant along each constraint (`c` for columns, `r-c` and `r+c` for the two diagonal directions) — avoiding an O(n) board scan on every single placement attempt.

10. Return-a-boolean to stop at the first solution (10)

Every earlier problem collects *all* valid results. Sudoku only needs *one* solved board, so `solve_sudoku`'s backtrack function returns `True/False` and the search short-circuits (`if backtrack(idx+1): return True`) the instant a full solution is found, instead of continuing to explore for alternatives that aren't needed. This return-a-boolean-to-short-circuit shape is worth recognizing as a distinct variant from the "collect everything" shape used everywhere else in this folder — it changes how the un-choose step is guarded (only undo if the recursive call *failed*).

Complexity summary

#	Problem	Time (worst case)	Extra space
01	subsets	$O(n \cdot 2^n)$	$O(n)$ recursion depth
02	subsets_ii	$O(n \cdot 2^n)$	$O(n)$
03	permutations	$O(n \cdot n!)$	$O(n)$

#	Problem	Time (worst case)	Extra space
04	combination_sum	$O(2^{\text{target}})$ rough bound	$O(\text{target})$
05	combination_sum_ii	$O(2^n)$	$O(n)$
06	generate_parentheses	$O(4^n / \sqrt{n})$ (Catalan number)	$O(n)$
07	letter_combinations_of_phone_number	$O(4^n)$	$O(n)$
08	palindrome_partitioning	$O(n \cdot 2^n)$	$O(n)$
09	n_queens	$O(n!)$ rough bound	$O(n)$
10	sudoku_solver	$O(9^{\text{empty cells}})$ worst case	$O(1)$ beyond the sets

These are inherently exponential problems — the whole point of the pruning tricks in patterns 3, 5, and 9 is to cut down the *constant factor and practical branching* of that exponential search, not to change its fundamental complexity class. Be ready to say this explicitly: backtracking problems are rarely "optimized to polynomial," they're "optimized to prune aggressively within an exponential search."

Common interview follow-ups to be ready for

- **subsets/permutations**: "can you generate them iteratively instead of recursively?" — subsets can be built iteratively (each existing subset doubles: with/without the next element); an iterative permutation generator (e.g. Heap's algorithm) is possible but far less natural to derive live than the recursive one.
- **combination_sum**: "what if negative numbers were allowed in candidates?" — the `remaining < 0` pruning stops working (a negative number could bring remaining back toward 0 later), and the search could become infinite with reuse allowed; discuss why the problem's constraints (positive candidates) make the pruning valid.
- **generate_parentheses**: "how many valid strings are there for a given n ?" — the n th Catalan number; knowing this names the complexity precisely instead of a vague "exponential."
- **n_queens**: "what if you only needed the *count* of solutions, not every board?" — drop the `board` reconstruction and the `result` list, just increment a counter at the base case; saves the $O(n)$ string-building cost per solution.
- **sudoku_solver**: "how would you speed this up further?" — pick the empty cell with the *fewest* remaining valid candidates first (most-constrained-variable heuristic) rather than processing empty cells in a fixed left-to-right order; dramatically prunes the search in practice even though worst-case complexity is unchanged.
- General: "what's the difference between backtracking and plain DFS?" — be ready with a one-sentence answer: backtracking is DFS over a space of *choices* with explicit undo, used to explore all valid combinations/arrangements/placements, versus DFS over a fixed graph/tree structure that already exists.

Self-check before moving on

You should be able to, without looking at the code:

- Write the choose/explore/un-choose skeleton from memory and adapt it live to a new problem statement.
- Explain the difference between a `start` index and a `used` array, and state which one a new problem needs just from reading whether it asks for combinations or permutations.
- Explain the `i > start` duplicate-skip guard and why it's not the same as just checking `nums[i] == nums[i-1]` unconditionally.
- State why N-Queens' three constraint sets each use a different key (`c`, `r-c`, `r+c`) and what invariant each key captures.
- Explain why Sudoku's backtrack function returns a boolean while every other problem in this folder returns nothing and just appends to a shared result list.

This is the first topic of Phase 2 (Algorithmic Patterns) per the README's syllabus, following Phase 1's 9 core data structure topics — next up is `11_graphs`.

Programs

01. Subsets

01_subsets.py

Explanation

Problem: Given an array of unique integers, return all possible subsets (the power set).

Source : LeetCode 78 - Subsets

Example:

Input: [1, 2, 3]

Output: [], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]

Idea: the template every problem in this folder is built on:
choose -> explore -> un-choose. **path** holds the subset being built;
every recursive call first records the **current** path as a valid subset (unlike permutations, a subset is valid at every partial length, not just when full), then tries adding each remaining candidate in turn, recursing, and removing it again before trying the next candidate ("un-choose" - this is what makes it backtracking instead of a single forward walk). **start** ensures each element is only ever considered **after** the ones already in path, which is what prevents [1,2] and [2,1] from both appearing (subsets don't care about order, so only one arrangement of each combination should ever be generated).

Code

```
def subsets(nums: list[int]) -> list[list[int]]:
    result = []
    path: list[int] = []

    def backtrack(start: int) -> None:
        result.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
    return result

def _normalize(subsets_list):
    return sorted(tuple(s) for s in subsets_list)

if __name__ == "__main__":
    tests = [
        ([1, 2, 3], [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]),
        ([], [[]]),
        ([0], [[], [0]]),
    ]
```

```

]

for i, (inp, expected) in enumerate(tests, 1):
    got = subsets(inp)
    status = "PASS" if _normalize(got) == _normalize(expected) else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

02. Subsets II

02_subsets_ii.py

Explanation

Problem: Given an integer array that may contain duplicates, return all possible subsets, with no duplicate subsets in the result.

Source : LeetCode 90 - Subsets II

Example:

Input: [1, 2, 2]

Output: [], [1], [1,2], [1,2,2], [2], [2,2]

Idea: same shape as 01, plus one guard against duplicate output. Sort first, so equal values become adjacent. At each level of the loop, skip a candidate if it's equal to the *previous* candidate at that same level (`i > start and nums[i] == nums[i-1]`) - the `i > start` check is what distinguishes "two equal values used together in one subset" (allowed - that's just consuming both 2s) from "two equal values both starting a sibling subset at this level" (not allowed - that would generate the same subset twice).

Code

```

def subsets_with_dup(nums: list[int]) -> list[list[int]]:
    nums = sorted(nums)
    result = []
    path: list[int] = []

    def backtrack(start: int) -> None:
        result.append(path[:])
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
    return result

def _normalize(subsets_list):
    return sorted(tuple(s) for s in subsets_list)

if __name__ == "__main__":
    tests = [
        ([1, 2, 2], [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]),
        ([0], [[], [0]]),
    ]

```

```

    ([2, 2], [[], [2], [2, 2]]),
]

for i, (inp, expected) in enumerate(tests, 1):
    got = subsets_with_dup(inp)
    status = "PASS" if _normalize(got) == _normalize(expected) else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

03. Permutations

03_permutations.py

Explanation

Problem: Given an array of unique integers, return all possible permutations (every arrangement, order matters).

Source : LeetCode 46 - Permutations

Example:

Input: [1, 2, 3]

Output: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]

Idea: unlike 01's subsets, order matters here, so there's no **start** index to prevent revisiting earlier elements - every still-unused element is a valid next choice at every position, tracked with a **used** array instead. A path only counts as a complete result once it reaches the full length (unlike subsets, a partial permutation isn't itself a valid answer - it must be recorded only at the leaves of the recursion tree, not at every node).

Code

```

def permute(nums: list[int]) -> list[list[int]]:
    result = []
    path: list[int] = []
    used = [False] * len(nums)

    def backtrack() -> None:
        if len(path) == len(nums):
            result.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            backtrack()
            path.pop()
            used[i] = False

    backtrack()
    return result

def _normalize(perms):
    return sorted(tuple(p) for p in perms)

if __name__ == "__main__":

```

```

tests = [
    ([1, 2, 3], [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]),
    ([0, 1], [[0, 1], [1, 0]]),
    ([5], [[5]]),
]

for i, (inp, expected) in enumerate(tests, 1):
    got = permute(inp)
    status = "PASS" if _normalize(got) == _normalize(expected) else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

04. Combination Sum

04_combination_sum.py

Explanation

Problem: Given distinct candidates and a target, return all unique combinations where the chosen numbers sum to target. The same number may be used an unlimited number of times.

Source : LeetCode 39 - Combination Sum

Example:

Input: candidates = [2, 3, 6, 7], target = 7

Output: [[2, 2, 3], [7]]

Idea: subsets-shaped backtracking (01's **start** index prevents using earlier elements again as *siblings*), but with the target sum as the stopping condition instead of subset-at-every-node, and with *reuse allowed* - the recursive call passes **i** (not **i + 1**) as the next start, meaning the same candidate can be picked again immediately after itself. Two base cases: **remaining == 0** is success (record the path), **remaining < 0** is failure (prune - no need to keep exploring a sum that already overshoot).

Code

```

def combination_sum(candidates: list[int], target: int) -> list[list[int]]:
    result = []
    path: list[int] = []

    def backtrack(start: int, remaining: int) -> None:
        if remaining == 0:
            result.append(path[:])
            return
        if remaining < 0:
            return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, remaining - candidates[i]) # i, not i+1: reuse allowed
            path.pop()

    backtrack(0, target)
    return result

def _normalize(combos):
    return sorted(tuple(sorted(c)) for c in combos)

```

```

if __name__ == "__main__":
    tests = [
        ([2, 3, 6, 7], 7, [[2, 2, 3], [7]]),
        ([2, 3, 5], 8, [[2, 2, 2, 2], [2, 3, 3], [3, 5]]),
        ([2], 1, []),
    ]

    for i, (candidates, target, expected) in enumerate(tests, 1):
        got = combination_sum(candidates, target)
        status = "PASS" if _normalize(got) == _normalize(expected) else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

05. Combination Sum II

05_combination_sum_ii.py

Explanation

Problem: Given a collection of candidates that may contain duplicates and a target, return all unique combinations that sum to target. Each number may be used at most once (no reuse).

Source : LeetCode 40 - Combination Sum II

Example:

Input: candidates = [10, 1, 2, 7, 6, 1, 5], target = 8

Output: [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]

Idea: combines 02's duplicate-skip trick with 04's target-sum backtracking, minus the reuse. Sort first so duplicates are adjacent and skip a candidate equal to the previous one *at the same tree level* (same reasoning as 02: this blocks generating the same combination twice, not blocks using two equal values together). The recursive call passes `i + 1` (not `i`, unlike 04) since each element can only be used once. One extra optimization: since candidates are sorted, the moment `candidates[i] > remaining`, every later candidate is even bigger, so the loop can `break` entirely instead of just `continue`-ing past this one option.

Code

```

def combination_sum2(candidates: list[int], target: int) -> list[list[int]]:
    candidates = sorted(candidates)
    result = []
    path: list[int] = []

    def backtrack(start: int, remaining: int) -> None:
        if remaining == 0:
            result.append(path[:])
            return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue
            if candidates[i] > remaining:
                break
            path.append(candidates[i])
            backtrack(i + 1, remaining - candidates[i])
            path.pop()

```

```

        backtrack(0, target)
        return result

def _normalize(combos):
    return sorted(tuple(c) for c in combos) # already in sorted order per combo

if __name__ == "__main__":
    tests = [
        ([10, 1, 2, 7, 6, 1, 5], 8, [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]),
        ([2, 5, 2, 1, 2], 5, [[1, 2, 2], [5]]),
    ]

    for i, (candidates, target, expected) in enumerate(tests, 1):
        got = combination_sum2(candidates, target)
        status = "PASS" if _normalize(got) == _normalize(expected) else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

06. Generate Parentheses

06_generate_parentheses.py

Explanation

Problem: Given n pairs of parentheses, generate all combinations of well-formed parenthesis strings.

Source : LeetCode 22 - Generate Parentheses

Example:

Input: $n = 3$

Output: ["((()))", "(())()", "(())()", "()(())", "()()()"]

Idea: rather than generating all $2^{(2n)}$ strings of parens and filtering for validity, prune invalid paths *during* construction - a much smaller search tree. Two counters, `open_count` and `close_count`, track how many of each have been placed. An open paren can be added any time `open_count < n` (haven't used them all yet). A close paren can only be added if `close_count < open_count` (there must be a still-unmatched open paren to close - this is exactly what keeps every prefix of the string valid, not just the final result). No explicit "un-choose" of the counters is needed since they're passed by value into each recursive call.

Code

```

def generate_parenthesis(n: int) -> list[str]:
    result = []
    path: list[str] = []

    def backtrack(open_count: int, close_count: int) -> None:
        if len(path) == 2 * n:
            result.append("".join(path))
            return
        if open_count < n:
            path.append("(")
            backtrack(open_count + 1, close_count)
            path.pop()

```

```

        if close_count < open_count:
            path.append("(")
            backtrack(open_count, close_count + 1)
            path.pop()

    backtrack(0, 0)
    return result

if __name__ == "__main__":
    tests = [
        (3, {"((())", "(()())", "()(())", "()()()", "(()()())"},
        (1, {"()"}),
    ]

    for i, (n, expected) in enumerate(tests, 1):
        got = set(generate_parenthesis(n))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

07. Letter Combinations Of Phone Number

07_letter_combinations_of_phone_number.py

Explanation

Problem: Given a string of digits 2-9, return all possible letter combinations the digits could represent on an old phone keypad (2 -> "abc", 3 -> "def", etc).

Source : LeetCode 17 - Letter Combinations of a Phone Number

Example:

Input: "23"

Output: ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]

Idea: the simplest possible backtracking shape - no pruning, no constraint to check, just a direct Cartesian product of "letters for digit 0" x "letters for digit 1" x ... generated via recursion instead of nested loops (useful because the number of digits, and therefore the nesting depth, isn't known ahead of time). At each recursion level, try every letter available for the current digit, recurse to the next digit, and un-choose before trying the next letter - same choose/explore/un-choose shape as every problem in this folder, just without any pruning condition.

Code

```

def letter_combinations(digits: str) -> list[str]:
    if not digits:
        return []

    mapping = {
        "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
        "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz",
    }
    result = []
    path: list[str] = []

```

```

def backtrack(index: int) -> None:
    if index == len(digits):
        result.append("".join(path))
        return
    for ch in mapping[digits[index]]:
        path.append(ch)
        backtrack(index + 1)
        path.pop()

backtrack(0)
return result

if __name__ == "__main__":
    tests = [
        ("23", {"ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"}),
        ("", set()),
        ("2", {"a", "b", "c"}),
    ]

    for i, (digits, expected) in enumerate(tests, 1):
        got = set(letter_combinations(digits))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

08. Palindrome Partitioning

08_palindrome_partitioning.py

Explanation

Problem: Given a string *s*, partition it so that every substring in the partition is a palindrome. Return all possible partitions.

Source : LeetCode 131 - Palindrome Partitioning

Example:

Input: "aab"

Output: [["a", "a", "b"], ["aa", "b"]]

Idea: backtracking over *where to cut*, not over which elements to pick. At each position **start**, try every possible end point for the next piece (**end** from start+1 to len(s)); only recurse into that choice if **s[start:end]** is itself a palindrome (pruning branches that could never lead to a valid full partition). A complete partition is found once **start** reaches the end of the string - every character has been assigned to some palindromic piece.

Code

```

def partition(s: str) -> list[list[str]]:
    result = []
    path: list[str] = []

    def is_palindrome(sub: str) -> bool:
        return sub == sub[::-1]

    def backtrack(start: int) -> None:
        if start == len(s):
            result.append(path[:])

```

```

        return
    for end in range(start + 1, len(s) + 1):
        substring = s[start:end]
        if is_palindrome(substring):
            path.append(substring)
            backtrack(end)
            path.pop()

    backtrack(0)
    return result

def _normalize(partitions):
    return sorted(tuple(p) for p in partitions)

if __name__ == "__main__":
    tests = [
        ("aab", [["a", "a", "b"], ["aa", "b"]]),
        ("a", [["a"]]),
        ("", [[]]),
    ]

    for i, (s, expected) in enumerate(tests, 1):
        got = partition(s)
        status = "PASS" if _normalize(got) == _normalize(expected) else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

09. N Queens

09_n_queens.py

Explanation

Problem: Place n queens on an $n \times n$ chessboard so that no two queens attack each other (no shared row, column, or diagonal). Return all distinct solutions, each as a list of strings representing the board.

Source : LeetCode 51 - N-Queens

Example:

Input: $n = 4$

Output: [

[".Q..", "...Q", "Q...", "..Q."],

["..Q.", "Q...", "...Q", ".Q.."],

]

Idea: place one queen per row (so row conflicts are structurally impossible - `placement[r]` is the column of the queen in row r , never two queens sharing a row by construction), and use three sets to track which columns and diagonals are already under attack:

`cols`, plus `diag1` for "/"-diagonals (constant $r - c$) and `diag2` for "\"-diagonals (constant $r + c$) - every cell on the same diagonal shares one of these two values, which is what makes $O(1)$ diagonal-conflict checks possible instead of scanning the board.

Trying every column at each row, skipping any that conflict, and un-marking all three sets on backtrack is the same choose/explore/

un-choose shape as every earlier problem, just with a richer constraint check.

Code

```
def solve_n_queens(n: int) -> list[list[str]]:
    result = []
    cols: set[int] = set()
    diag1: set[int] = set() # r - c, constant along "/" diagonals
    diag2: set[int] = set() # r + c, constant along "\" diagonals
    placement: list[int] = []

    def backtrack(row: int) -> None:
        if row == n:
            board = [ "." * c + "Q" + "." * (n - c - 1) for c in placement ]
            result.append(board)
            return
        for c in range(n):
            if c in cols or (row - c) in diag1 or (row + c) in diag2:
                continue
            cols.add(c)
            diag1.add(row - c)
            diag2.add(row + c)
            placement.append(c)

            backtrack(row + 1)

            placement.pop()
            cols.remove(c)
            diag1.remove(row - c)
            diag2.remove(row + c)

    backtrack(0)
    return result

if __name__ == "__main__":
    tests = [
        (4, 2),
        (1, 1),
        (2, 0),
        (8, 92),
    ]

    for i, (n, expected_count) in enumerate(tests, 1):
        got = solve_n_queens(n)
        status = "PASS" if len(got) == expected_count else "FAIL"
        print(f"Test {i}: {status} (n={n}, got {len(got)} solutions, expected {expected_count})")
```

10. Sudoku Solver

10_sudoku_solver.py

Explanation

Problem: Given a partially filled 9x9 Sudoku board ('.' for empty cells), fill it in place so every row, column, and 3x3 box contains the digits 1-9 exactly once.

Source : LeetCode 37 - Sudoku Solver

Example:

Input: a partially filled valid board

Output: the board filled in, modified in place

Idea: the hardest problem in this folder, but structurally still the same choose/explore/un-choose shape as every other one here - just with a 2D grid and three simultaneous constraints instead of one. Track which digits are already used in each row, column, and 3x3 box with sets (same O(1)-conflict-check idea as 09's diagonal sets), precomputed once from the board's initial filled cells. Recurse over only the empty cells (found once up front, not re-scanned every call): at each one, try every digit 1-9 not already conflicting in that cell's row/column/box, place it, recurse to the next empty cell, and undo the placement if no digit leads to a full solution. Returning **True/False** from the recursion (instead of collecting **all** solutions like 01-08) lets the search stop immediately once one full solution is found, since Sudoku only asks for one.

Code

```
def solve_sudoku(board: list[list[str]]) -> None:
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    empties = []

    for r in range(9):
        for c in range(9):
            val = board[r][c]
            if val == ".":
                empties.append((r, c))
            else:
                rows[r].add(val)
                cols[c].add(val)
                boxes[(r // 3) * 3 + c // 3].add(val)

    def backtrack(idx: int) -> bool:
        if idx == len(empties):
            return True

        r, c = empties[idx]
        b = (r // 3) * 3 + c // 3

        for digit in "123456789":
            if digit in rows[r] or digit in cols[c] or digit in boxes[b]:
                continue

            board[r][c] = digit
            rows[r].add(digit)
            cols[c].add(digit)
            boxes[b].add(digit)

            if backtrack(idx + 1):
                return True

            board[r][c] = "."
            rows[r].remove(digit)
            cols[c].remove(digit)
            boxes[b].remove(digit)
```

```

        return False

    backtrack(0)

if __name__ == "__main__":
    board = [
        ["5", "3", ".", ".", "7", ".", ".", ".", "."],
        ["6", ".", ".", "1", "9", "5", ".", ".", "."],
        [".", "9", "8", ".", ".", ".", ".", "6", "."],
        ["8", ".", ".", ".", "6", ".", ".", ".", "3"],
        ["4", ".", ".", "8", ".", "3", ".", ".", "1"],
        ["7", ".", ".", "2", ".", ".", ".", ".", "6"],
        [".", "6", ".", ".", ".", ".", "2", "8", "."],
        [".", ".", ".", "4", "1", "9", ".", ".", "5"],
        [".", ".", ".", ".", "8", ".", ".", "7", "9"],
    ]
    expected = [
        ["5", "3", "4", "6", "7", "8", "9", "1", "2"],
        ["6", "7", "2", "1", "9", "5", "3", "4", "8"],
        ["1", "9", "8", "3", "4", "2", "5", "6", "7"],
        ["8", "5", "9", "7", "6", "1", "4", "2", "3"],
        ["4", "2", "6", "8", "5", "3", "7", "9", "1"],
        ["7", "1", "3", "9", "2", "4", "8", "5", "6"],
        ["9", "6", "1", "5", "3", "7", "2", "8", "4"],
        ["2", "8", "7", "4", "1", "9", "6", "3", "5"],
        ["3", "4", "5", "2", "8", "6", "1", "7", "9"],
    ]

    solve_sudoku(board)
    status = "PASS" if board == expected else "FAIL"
    print(f"Test 1: {status}")
    if status == "FAIL":
        for row in board:
            print(row)

```